claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-ab19bc7c69e2f8bfd.jsonl`
- SHA-256: `517cdd043828a70b39472eb0044d0f2678014b533feb7c15f5e8b7194373c966`
- Source modified: `2026-07-06T17:29:57+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:26:48.057Z)

```
You are a CORRECTNESS reviewer.
REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
DIFF UNDER REVIEW (read this first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff

GOAL OF THE CHANGE (wf294 incident): the Cloud Run rally-worker reported a run as
status="success" with 0 segments even when the WASB shuttle-trajectory detector TIMED OUT
(1800s) and produced zero trajectory. A detector timeout/abort was indistinguishable from a
legitimately 0-rally video. The fix must make "detector failed/timed out" surface as a
failed status (NOT "success") in report.json AND the completion marker, while a genuine
0-rally match stays a success.

WHAT THE FIX DID (4 files, all in the diff):
1. backend/pipeline/segmenters/trajectory_hybrid.py ? process_video() now returns a
   backend.results.Failure (instead of a bare []) on TWO detector-failure paths: the env
   healthcheck-not-ready path, and the "run_predict produced no trajectory" abort path.
   A trajectory-produced-but-zero-windows case still returns bare [] (a legit 0-rally).
   Return annotation widened to Union[List[Dict[str,Any]], Failure]; kept the
   # type: ignore[override].
2. backend/worker/__main__.py ? added _serialize_and_push_outputs() helper (writes+pushes
   intervals.csv + report.json). _fail(rc) now writes+pushes a status="failed" report.json /
   intervals.csv (0 segments), best-effort, BEFORE the M6 done-marker. The success path uses
   the same helper. The segmenter-Failure branch already routed to _fail(3).
3. tests/test_tracknet_hybrid.py ? the two abort tests now assert isinstance(res, Failure);
   added a test that zero-windows stays a bare [] (not a Failure).
4. tests/test_worker.py ? new test that a segmenter Failure -> rc 3 + report.json
   status="failed" + done-marker returncode 3/status="failed"; strengthened the 0-segment
   test to assert status="success".

KEY CONSUMER CODE TO CROSS-CHECK (read the real files, do not assume):
- backend/worker/__main__.py: cmd_infer (the full control flow: validation -> segmenter ->
  isinstance(result, Failure) -> _fail(3); the finally: emit_indexer_report; the success tail),
  _write_done_marker, _write_report_json, _write_intervals_csv.
- backend/orchestration.py: cached_process_result (~line 288, requires report status=="success"
  to hydrate), probe_process_cache (~line 255), _read_bucket_json, _resume_one_remote_job usage.
- backend/compute/cloud_run.py: CloudRunCompute.run_infer (reads the done-marker returncode +
  outputs/<id>/report.json).
- backend/api/job_worker.py: _resume_one_remote_job (waits for report.json to EXIST, then reads
  status; marks the job failed if status != success), _run_claimed_job.
- backend/results.py: Success/Failure/Result/unwrap_or_failure contract.
- backend/main.py (isinstance Failure branch ~1557), backend/pipeline/segmenters/gemini.py &
  motion.py (the existing Failure precedent), yolo_hybrid.py (a PARALLEL hybrid segmenter).
```

CONSTRAINTS THE FIX MUST RESPECT: ruff clean, mypy clean (trajectory_hybrid is NOT quarantined; worker.__main__ IS quarantined via mypy.ini ignore_errors), full suite green, coverage floor 70%.
The full suite is already green (1736 passed) and ruff/mypy clean ? so do NOT report "tests might fail" speculatively; report only concrete defects you can point at in the code.

Only report REAL defects: a wrong/edge behavior, a silent-failure path still leaking as success, a control-plane interaction regression, an idempotency/concurrency bug, a broken test assertion, or a genuine convention violation. Cite exact file:line. If a concern is speculative or you cannot ground it in the actual code, DO NOT report it.

YOUR LENS: Does the fix actually surface every detector timeout/abort as a failure, with no remaining leak into status="success"? Trace EVERY return path of trajectory_hybrid.process_video and every branch of worker.cmd_infer. Check specifically:
- Does the Failure returned by process_video reach worker's isinstance(result, Failure) and route to _fail(3)? Is the same true in main.py and orchestration.py?
- The line "segments = result.value if hasattr(result, 'value') else result" ? could a Failure ever slip past the isinstance check? Could a Success wrapper or a bare list mis-unwrap?
- In _fail(), the report/marker are written from a closure over video_id/scratch/storage_cfg/args ? are all defined at every _fail call site (validation rc2, unknown-segmenter rc2, Failure rc3)?
- Is the "finally: emit_indexer_report" ordering vs _fail's push safe? Any double-write or path where a Failure still ends up writing status="success"?
- Does the 0-segment "loud diagnostic" warning still fire correctly for the LEGIT empty case and NOT for the failure case?
Report concrete correctness defects only, with file:line and a failure scenario.

2. assistant (2026-07-06T17:26:53.369Z)

I'll start by reading the diff under review, then cross-check against the real consumer code.

3. user (2026-07-06T17:26:55.098Z)

```
1       diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2       index 503f6d8..f96b495 100644
3       --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4       +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5       @@ -22,7 +22,7 @@ import concurrent.futures
6        import logging
7        import os
8        import time
9       -from typing import Any, Dict, List, Optional, Tuple
10      +from typing import Any, Dict, List, Optional, Tuple, Union
11
12       import cv2
13
14      @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15       from backend.pipeline.detectors.base import DetectorRunner
16       from backend.pipeline.segmenters.base import BasePlaySegmenter
17       from backend.providers import provider_registry
18      +
19      +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20      +from backend.results import Failure
21       from backend.sports import sport_registry
22       from backend.utils.paths import output_base
23       from backend.utils.video import extract_video_chunk, get_video_duration
24      @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25                video_path: str,
26                player_pool: Optional[List[str]] = None,
27                max_frames: Optional[int] = None,
28      -    ) -> List[Dict[str, Any]]:
29      -        # override-ignore: the ABC declares the Result contract (Success|Failure)
30      -        # but every live segmenter still returns a bare list ? the documented
```

```
-        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
+    ) -> "Union[List[Dict[str, Any]], Failure]":
+        # Partial adoption of the Result contract (the ABC declares Success|Failure).
+        # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
+        # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
+        # genuine 0-rally video (which still returns a bare ``[]``). Every caller
+        # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
+        # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
                label = self.display_label
                # --- Sport profile + AI provider wiring (identical across segmenters) ---
                sport_name = self.config.sport
    @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                ok, msg = runner.healthcheck()
                if not ok:
                    logger.error("%s detector environment not ready: %s", label, msg)
-                   return []
+                   # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
+                   # failed/retryable instead of writing an empty-but-"successful" result.
+                   return Failure(
+                       message=f"{label} detector environment not ready: {msg}",
+                       is_recoverable=True,
+                   )
                logger.info("%s detector env OK: %s", label, msg)

                # --- Phase 2: trajectory -> candidate rally windows ---
    @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
                if not csv_path:
                    logger.error("%s produced no trajectory; aborting.", label)
-                   return []
+                   # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
+                   # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
+                   # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
+                   # rather than a silent 0-segment "success" (the wf294 incident).
+                   return Failure(
+                       message=(
+                           f"{label} detector produced no trajectory (timed out or aborted) ?
"
+                           "detector failure, not a 0-rally video."
+                       ),
+                       is_recoverable=True,
+                   )

                points = runner.parse_trajectory_csv(csv_path)
                logger.info(
    diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
    index 461d7ca..88c50f6 100644
    --- a/backend/worker/__main__.py
    +++ b/backend/worker/__main__.py
    @@ -114,6 +114,42 @@ def _write_report_json(
                )


    +def _serialize_and_push_outputs(
    +    scratch: str,
    +    out_uri: str,
    +    video_id: str,
```

```
87      +    segments: List[dict],
88      +    status: str,
89      +    storage_cfg: dict,
90      +    video_uri: str = "",
91      +) -> List[str]:
92      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94      +
95      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98      +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
100     +    out_local = os.path.join(scratch, "outputs", video_id)
101     +    os.makedirs(out_local, exist_ok=True)
102     +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103     +    _write_report_json(
104     +        video_id,
105     +        segments,
106     +        status,
107     +        os.path.join(out_local, "report.json"),
108     +        video_uri=video_uri,
109     +    )
110     +    out_prefix = out_uri.rstrip("/")
111     +    pushed: List[str] = []
112     +    for fn in sorted(os.listdir(out_local)):
113     +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114     +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115     +        pushed.append(uri)
116     +    return pushed
117     +
118     +
119      def _run_startup_checks(config: Any) -> bool:
120          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121          best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122     @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123          )
124
125          def _fail(rc: int) -> int:
126     -        # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127     -        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128     -        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129     +        # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130     +        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131     +        # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132     +        # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133     +        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134     +        # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135     +        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
```

```
136     +        try:
137     +            _serialize_and_push_outputs(
138     +                scratch,
139     +                args.out_uri,
140     +                video_id,
141     +                [],
142     +                "failed",
143     +                storage_cfg,
144     +                str(getattr(args, "video_uri", "") or ""),
145     +            )
146     +        except Exception as e:  # never mask the real failure on an artifact-push
hiccup
147     +            logger.warning("[worker] could not push failure artifacts: %s", e)
148     +        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149              if getattr(args, "done_uri", None):
150                  _write_done_marker(
151                      args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152      @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153              )
154
155          # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156      -    out_local = os.path.join(scratch, "outputs", video_id)
157      -    os.makedirs(out_local, exist_ok=True)
158      -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159      -    _write_report_json(
160      +    pushed = _serialize_and_push_outputs(
161      +        scratch,
162      +        args.out_uri,
163              video_id,
164              segments,
165              status,
166      -        os.path.join(out_local, "report.json"),
167      -        video_uri=str(getattr(args, "video_uri", "") or ""),
168      +        storage_cfg,
169      +        str(getattr(args, "video_uri", "") or ""),
170          )
171
172      -    out_prefix = args.out_uri.rstrip("/")
173      -    pushed = []
174      -    for fn in sorted(os.listdir(out_local)):
175      -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176      -        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177      -        pushed.append(uri)
178      -
179          print(
180              f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181          )
182      diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183      index 64909e7..c5ae442 100644
184      --- a/tests/test_tracknet_hybrid.py
185      +++ b/tests/test_tracknet_hybrid.py
186      @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187       from unittest.mock import MagicMock, patch
188
189       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190      +from backend.results import Failure
191
192
193       def _window(start=1.0, end=4.0):
194      @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197          res = seg.process_video("v1", "clip.mp4")
```

```
    -    assert res == []
    +    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
    +    # instead of writing an empty "success".
    +    assert isinstance(res, Failure)
    +    assert "environment not ready" in res.message
         # Should bail before ever running inference.
         mock_runner_cls.return_value.run_predict.assert_not_called()

    @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
         mock_provider_registry.get.return_value = MagicMock()

         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
    -    assert seg.process_video("v1", "clip.mp4") == []
    +    res = seg.process_video("v1", "clip.mp4")
    +    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
    +    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
    +    assert isinstance(res, Failure)
    +    assert "no trajectory" in res.message and res.is_recoverable
    +
    +
    +@_patched
    +def test_pipeline_zero_windows_is_legit_empty_not_failure(
    +    mock_runner_cls,
    +    mock_probe,
    +    mock_dur,
    +    mock_extract,
    +    mock_provider_registry,
    +    mock_env,
    +):
    +    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
    +    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
    +    (0-rally "success"), NOT a Failure."""
    +    mock_env.return_value = "dummy_key"
    +    mock_dur.return_value = 30.0
    +    mock_extract.return_value = True
    +    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
    +    mock_provider_registry.get.return_value = MagicMock()
    +
    +    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
    +    res = seg.process_video("v1", "clip.mp4")
    +    assert res == [] and not isinstance(res, Failure)
    +    # The detector DID run (a genuine empty, not an abort).
    +    mock_runner_cls.return_value.run_predict.assert_called_once()


     @_patched
    diff --git a/tests/test_worker.py b/tests/test_worker.py
    index 96edbee..63cc18e 100644
    --- a/tests/test_worker.py
    +++ b/tests/test_worker.py
    @@ -10,6 +10,7 @@ import json
     import pytest

     from backend.config import load_config
    +from backend.results import Failure
     from backend.worker import __main__ as wmain

     # ------------------------------------------------------------- pure helpers
    @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
```

```
257
258
259      class _EmptySeg:
260      -    """A segmenter that finds nothing ? the cold-start failure mode."""
261      +    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare []).."""
262
263           def __init__(self, db, config):
264               pass
265      @@ -200,6 +201,20 @@ class _EmptySeg:
266               return []
267
268
269      +class _FailSeg:
270      +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271      +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272      +
273      +    def __init__(self, db, config):
274      +        pass
275      +
276      +    def process_video(self, video_id, path, max_frames=None):
277      +        return Failure(
278      +            message="WASB detector produced no trajectory (timed out or aborted)",
279      +            is_recoverable=True,
280      +        )
281      +
282      +
283       def test_setup_logging_levels():
284           import logging
285
286      @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287           assert "0 segments" in msgs and "vidZ" in msgs
288           assert "detector_impl=native" in msgs  # surfaces the resolved detector
289           assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
290      -    assert (
291      -        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292      -            "segment_count"
293      +    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294      +    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295      +    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296      +    assert report["segment_count"] == 0 and report["status"] == "success"
297      +
298      +
299      +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300      +    tmp_path, monkeypatch
301      +):
302      +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303      +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304      +    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305      +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306      +    monkeypatch.setattr(
307      +        wmain.validator_registry,
308      +        "run_all_with_context",
309      +        lambda path, cfg: ([_OKResult()], {}),
310      +    )
311      +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312      +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313      +    video = tmp_path / "in.mp4"
314      +    video.write_bytes(b"FAKE")
```

```
315     +    out = tmp_path / "bucket"
316     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317     +
318     +    rc = wmain.main(
319     +        [
320     +            "infer",
321     +            "--video-uri",
322     +            str(video),
323     +            "--out-uri",
324     +            str(out),
325     +            "--scratch",
326     +            str(tmp_path / "s"),
327     +            "--segmenter",
328     +            "wasb_hybrid",
329     +            "--done-uri",
330     +            str(done),
331              ]
332     -        == 0
333          )
334     +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335     +    assert rc == 3
336     +    # report.json is written as an explicit FAILURE, not a false empty "success".
337     +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338     +    assert report["status"] == "failed" and report["segment_count"] == 0
339     +    assert report["segments"] == []
340     +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341     +    marker = json.loads(done.read_text())
342     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
343     +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346      def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347
```

4. assistant (2026-07-06T17:26:57.614Z)

I'll trace the actual consumer code to verify the fix is complete. Let me read the key files.

5. user (2026-07-06T17:26:58.975Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
```

```
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
62
63      def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64          """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66          - ``Failure``            -> ``(None, failure)``
67          - ``Success(value=v)``   -> ``(v, None)``
68          - a bare value (legacy code that predates this contract and still returns a raw
list)
69            -> ``(value, None)``
70
71          This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72          that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73          on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74          (Success | Failure | legacy bare value)."""
75          if isinstance(result, Failure):
76              return None, result
77          value = result.value if hasattr(result, "value") else result
78          return value, None
79
```

6. user (2026-07-06T17:26:58.990Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""

from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("backend.worker")


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v


```

```
    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
```

```
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
```

```
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
        poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
        timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
        )
        try:
            result = compute.run_infer(spec)
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
```

```
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(
            local_video, config
        )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
```

```
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
```

```
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
```

```
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess

        try:
            out = subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
```

```
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
```

```
shell out
512         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
513
514         Two trajectory sources (the train pipeline + harness are identical either way):
515           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
516             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
517           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
518             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
519             is the M3.5 "first real training" path; only the trajectory source flips.
520         """
521         import subprocess
522
523         config = load_config(args.config) if args.config else load_config()
524         _apply_overrides(config, args.set)
525         if not _run_startup_checks(config):
526             return 2
527         storage_cfg = config.storage.model_dump()
528
529         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
530         labels_dir = os.path.join(scratch, "labels")
531         traj_dir = os.path.join(scratch, "trajectories")
532         videos_dir = os.path.join(scratch, "videos")
533         os.makedirs(labels_dir, exist_ok=True)
534         os.makedirs(traj_dir, exist_ok=True)
535
536         corpus = args.corpus_uri.rstrip("/")
537         label_uris = sorted(
538             u
539             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
540             if u.lower().endswith(".csv")
541         )
542         if len(label_uris) < 2:
543             print(
544                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
545                 f"under {corpus}/labels/"
546             )
547             return 2
548
549         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
550         runner = None
551         video_by_stem: dict = {}
552         if args.trajectory_source == "detector":
553             os.makedirs(videos_dir, exist_ok=True)
554             runner = _resolve_train_detector(args, config)
555             if runner is None:
556                 return 2
557             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558                 vname = storage.parse_uri(vuri).name
559                 if not vname.lower().endswith(_VIDEO_EXTS):
560                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561                 video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563         print(f"[worker] trajectory source: {args.trajectory_source}")
564         specs: List[dict] = []
565         for uri in label_uris:
566             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
567             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
568             local_label = storage.fetch(
```

```
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
```

```
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

```

```
        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
```

```
758         pt.add_argument(
759             "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
760         )
761         pt.add_argument("--config", default=None)
762         pt.add_argument("--scratch", default=None)
763         pt.add_argument(
764             "--fps",
765             type=float,
766             default=30.0,
767             help="synth trajectory fps (detector probes the video)",
768         )
769         pt.add_argument(
770             "--frame-width",
771             type=float,
772             default=1280.0,
773             help="synth trajectory frame width (detector probes the video)",
774         )
775         pt.add_argument("--set", action="append", default=[], metavar="k=v")
776         pt.set_defaults(func=cmd_train)
777         return p
778
779
780     def _setup_logging(verbose: bool) -> None:
781         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
782         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
783         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
784         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
785         entry), not at import, so importing the module never reconfigures a caller's
logging."""
786         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
787         # configured logging at import), which would leave the level at WARNING and re-
suppress the
788         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
789         logging.basicConfig(
790             level=logging.DEBUG if verbose else logging.INFO,
791             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
792             datefmt="%H:%M:%S",
793             force=True,
794         )
795
796
797     def main(argv=None) -> int:
798         args = build_parser().parse_args(argv)
799         _setup_logging(getattr(args, "verbose", False))
800         return args.func(args)
801
802
803     if __name__ == "__main__":
804         sys.exit(main())
805
```

7. assistant (2026-07-06T17:27:01.922Z)

Now let me look at the trajectory_hybrid.py return paths and the consumer code in main.py and orchestration.py.

8. user (2026-07-06T17:27:03.156Z)

```
260             ai_max_workers = idx_cfg.ai_max_workers
```

```
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
                # failed/retryable instead of writing an empty-but-"successful" result.
                return Failure(
                    message=f"{label} detector environment not ready: {msg}",
                    is_recoverable=True,
                )
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join(output_base(self.config), self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
                # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
                # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
                # rather than a silent 0-segment "success" (the wf294 incident).
                return Failure(
                    message=(
                        f"{label} detector produced no trajectory (timed out or aborted) ? "
                        "detector failure, not a 0-rally video."
                    ),
                    is_recoverable=True,
                )

            points = runner.parse_trajectory_csv(csv_path)
            logger.info(
                "Parsed %d trajectory points (%d visible).",
                len(points),
                sum(1 for p in points if p.visible),
            )

            all_candidate_windows = runner.trajectory_to_action_windows(
                points,
                fps=fps,
                frame_width=frame_width,
                chunk_start=0.0,
                velocity_thresh=velocity_thresh,
                merge_gap=merge_gap,
                min_window_duration=min_window_duration,
                chunk_index=0,
                max_window_duration=max_window_duration,
                min_split_gap=window_min_split_gap,
```

```
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                inactivity_temporal_density=inactivity_temporal_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info(
                "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                label,
                time.time() - t_start,
                len(all_candidate_windows),
            )

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(
                        f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                        f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
                    )

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(
                    cw, points, fps, frame_width, video_path, idx_cfg
                )
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id,
                        detector=self.name,
                        start_time=cw["start"],
                        end_time=cw["end"],
                        chunk_index=cw["chunk_index"],
                        confidence=min(1.0, cw["peak_velocity"]),
                        stats=window_stats[i],
                        detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(
                all_candidate_windows, window_stats, idx_cfg
            )

            ai_segments: List[dict] = []
            if skip_ai:
```

```
382                 # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
383                 # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
384                 # falls back to the duration-based estimate (no AI exchange count).
385                 ai_segments = [
386                     {
387                         "start_time": all_candidate_windows[wi]["start"],
388                         "end_time": all_candidate_windows[wi]["end"],
389                         "shuttle_exchange_count": None,
390                         "shot_count_confidence": "LocalNoAI",
391                         "_candidate_id": candidate_ids[wi],
392                     }
393                     for wi in handoff_indices
394                 ]
395                 logger.info(
396                     "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
397                     "(no AI handoff).",
398                     len(ai_segments),
399                 )
400             else:
401                 handoff_dir = os.path.join(
402                     output_base(self.config), f"chunks_{provider_name}_handoff"
403                 )
404                 os.makedirs(handoff_dir, exist_ok=True)
405                 logger.info(
406                     "Phase 3: %s validation on %d candidate windows...",
407                     provider_name,
408                     len(handoff_indices),
409                 )
410
411                 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
412                     w_start = max(0, window["start"] - handoff_padding)
413                     w_end = min(video_duration, window["end"] + handoff_padding)
414                     w_dur = w_end - w_start
415                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
416                     if not extract_video_chunk(
417                         video_path, w_start, w_dur, w_path, reencode=True
418                     ):
419                         return []
420                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
421                     segments, metrics = provider.analyze_video(
422                         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
423                     )
424                     # B2: a provider failure returns [] with metrics["error"] set ? that is
425                     # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
426                     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
427                     if metrics.get("error"):
428                         logger.warning(
429                             "[Handoff %d] provider error ? window %d skipped: %s",
430                             wi + 1,
431                             wi,
432                             metrics["error"],
433                         )
434                     valid = []
435                     for seg in segments:
436                         try:
437                             seg["start_time"] = float(seg["start_time"]) + w_start
438                             seg["end_time"] = float(seg["end_time"]) + w_start
439                             seg["_candidate_id"] = candidate_ids[wi]
```

9. assistant (2026-07-06T17:27:06.698Z)

Now I have the two Failure paths and the `if not all_candidate_windows: return []` bare-list path. Let me look at the rest of process_video (the tail return) and then the main.py and orchestration consumers.

10. user (2026-07-06T17:27:08.414Z)

```
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning(
                                "Dropping segment with unparseable timestamps: %s ? %s",
                                seg,
                                e,
                            )
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(
                    max_workers=ai_max_workers
                ) as executor:
                    futures = [
                        executor.submit(
                            process_candidate_window, wi, all_candidate_windows[wi]
                        )
                        for wi in handoff_indices
                    ]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info(
                    "Phase 3 completed. AI confirmed %d semantic play segments.",
                    len(ai_segments),
                )

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
                    )
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
```

```
499                                 "duration": dur,
500                                 "metadata": metadata,
501                             }
502                         )
503                 except Exception as e:
504                     logger.warning("Skipping invalid segment struct: %s", e)
505
506             return segments_data
507
```

11. assistant (2026-07-06T17:27:12.085Z)

Now the three return paths of process_video are clear: two `Failure` (healthcheck, no-trajectory), one bare `[]` at zero-windows, and one bare `segments_data` list at the tail. Now let me check the main.py and orchestration.py consumers, plus yolo_hybrid for parallel context.

12. user (2026-07-06T17:27:13.093Z)

```
1530                     print("\n" + "=" * 40)
1531                     print("VALIDATION SUMMARY")
1532                     print("=" * 40)
1533                     for r in val_results:
1534                         status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535                         print(f"{status_symbol} {r.validator_name}: {r.message}")
1536                     print("=" * 40 + "\n")
1537
1538                     if failures:
1539                         print("[FAIL] Video failed checks. Indexing halted.")
1540                         return
1541
1542                     # Index
1543                     segmenter_cls = segmenter_registry.get(seg_name)
1544                     if not segmenter_cls:
1545                         print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546                         return
1547
1548                     print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549                     segmenter_actual = seg_name
1550                     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551                     segmenter = segmenter_cls(db_instance, global_config)
1552                     result = segmenter.process_video(
1553                         video_id, local_video, max_frames=args.max_frames
1554                     )
1555                     segmentation_ran = True
1556
1557                     if isinstance(result, Failure):
1558                         print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559                         print("[FAIL] Indexing halted for video.")
1560                         segments = []
1561                     else:
1562                         segments = result.value if hasattr(result, "value") else result
1563                         print(
1564                             f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565                         )
1566                     # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567                     _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568                 finally:
1569                     # Always emit the per-video observability report. Never raises.
1570                     paths = emit_indexer_report(
1571                         db=db_instance,
1572                         video_id=video_id,
1573                         # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
```

```
1574                         # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                         # in DB provenance via db.add_video.
1576                         video_path=local_video,
1577                         config=_report_config_for_input(global_config, args.video_path),
1578                         val_results=val_results,
1579                         val_context=val_context,
1580                         segmenter_requested=args.segmenter,
1581                         segmenter_actual=segmenter_actual,
1582                         was_reingest=was_reingest,
1583                         segmentation_ran=segmentation_ran,
1584                     )
1585                     if paths:
1586                         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587                         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588                         print(f"[REPORT] Machine JSON     : {paths.get('json')}")
1589             finally:
1590                 storage.cleanup_acquired_local_input(
1591                     args.video_path, local_video, storage_cfg
1592                 )
1593             return
1594
1595         # Start Local Server Mode
1596         if args.app or args.server or not args.video_path:
1597             if args.app:
1598                 # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599                 # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
```

13. user (2026-07-06T17:27:14.122Z)

```
240
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
243     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
244     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
245     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
246     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
247     _REHYDRATE_LOCKS_GUARD = threading.Lock()
248
249
250     def _rehydrate_lock(video_id: str) -> threading.Lock:
251         with _REHYDRATE_LOCKS_GUARD:
252             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
257
258         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261         errors report False (? the normal dispatch path with its guards)."""
262         try:
263             existing = db.get_video(video_id)
264             if (
265                 existing
266                 and existing.get("status") == "processed"
267                 and db.query_play_segments(video_id, {})
```

```
            ):
                return True
            sc = _storage_settings(cfg)
            if not sc or (sc.get("backend") or "local") == "local":
                return False
            output_root = (sc.get("output_root") or "").rstrip("/")
            if not output_root:
                return False
            report_uri = f"{output_root}/outputs/{video_id}/report.json"
            if not storage.exists(report_uri, config=sc):
                return False
            report = _read_bucket_json(report_uri, sc)
            return bool(report and report.get("status") == "success")
        except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
            logger.warning(
                "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
            )
            return False


    def cached_process_result(
        db, cfg, video_id: str, video_path: str, requested_compute=None
    ) -> Optional[Dict[str, Any]]:
        """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
        the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
        re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
        source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
        completed work found (or the check failed) ? the caller dispatches normally. The
bucket
        branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
        partial earlier ingest is replaced, never double-counted.

        Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
        per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
        in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
        job before invoking (writes happen post-claim)."""
        try:
            with _rehydrate_lock(video_id):
                existing = db.get_video(video_id)
                if existing and existing.get("status") == "processed":
                    segs = db.query_play_segments(video_id, {})
                    if segs:
                        return _cached_process_payload(
                            video_id,
                            existing.get("validation_results", []),
                            segs,
                            requested_compute,
                        )
                sc = _storage_settings(cfg)
                if not sc or (sc.get("backend") or "local") == "local":
                    return None
                output_root = (sc.get("output_root") or "").rstrip("/")
                if not output_root:
```

14. assistant (2026-07-06T17:27:20.085Z)

Let me see the rest of `cached_process_result` (the bucket branch requiring status=="success") and the job_worker restart-recovery path.

15. user (2026-07-06T17:27:21.660Z)

```
318                 output_root = (sc.get("output_root") or "").rstrip("/")
319                 if not output_root:
320                     return None
321                 outputs_uri = f"{output_root}/outputs/{video_id}"
322                 report_uri = f"{outputs_uri}/report.json"
323                 if not storage.exists(report_uri, config=sc):
324                     return None
325                 report = _read_bucket_json(report_uri, sc)
326                 if not report or report.get("status") != "success":
327                     return None
328                 if not video_path:
329                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330                     # carries the worker's --video-uri (P3); older reports fall back to the
331                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332                     # recover, and the next same-md5 submit re-binds the truthful path via
333                     # add_video's upsert.
334                     video_path = str(report.get("video_uri") or "") or _first_object_under(
335                         f"{output_root}/videos/{video_id}/", sc
336                     )
337                 play_wm, cand_wm = db.segment_watermarks(video_id)
338                 db.add_video(video_id, video_path, "processed", [])
339                 n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340                 if n <= 0:
341                     return None
342                 segments = [
343                     s
344                     for s in db.query_play_segments(video_id, {})
345                     if int(s.get("id", 0)) > play_wm
346                 ]
347                 _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348                 if not segments:
349                     return None
350                 logger.info(
351                     "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352                     n,
353                     video_id,
354                     outputs_uri,
355                 )
356                 return _cached_process_payload(video_id, [], segments, requested_compute)
357         except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358             logger.warning(
359                 "[API] submit-time cache check failed for %s ? dispatching normally",
360                 video_id,
361                 exc_info=True,
362             )
363             return None
364 
365 
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception:  # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372 
373 
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375 
376 
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
```

16. user (2026-07-06T17:27:22.195Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import json
20      import logging
21      import os
22      import threading
23      import traceback
24      from concurrent.futures import ThreadPoolExecutor
25      from typing import Any, Dict, Optional
26
27      logger = logging.getLogger(__name__)
28
29      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32      # already parallelize their AI handoff internally via their own pools.
33      # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34      # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35      # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36      # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37      # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38      # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39      #
40      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44      #   - detector timeout_sec kills hung WSL/GPU calls
45      #   - Cloud Run poll has a max_wait_sec bound
46      #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47      # A future slice should add a per-job wall-clock timeout (the executor itself
48      # cannot enforce timeouts on the Thread).
49      _job_executor: Optional[ThreadPoolExecutor] = None
50      # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51      # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52      # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53      _drain_max_workers: int = 1
```

```
54
55
56      def _cloud_dispatch_possible(config: Any = None) -> bool:
57          """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58          ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59          wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60          orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61          not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62          dep = getattr(config, "deployment", None)
63          if getattr(dep, "compute_target", "") == "cloud_run":
64              return True
65          if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66              stor = getattr(config, "storage", None)
67              if (getattr(stor, "output_root", "") or "").strip():
68                  return True
69          return False
70
71
72      def _resolve_drain_workers(config: Any = None) -> int:
73          """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
74          may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75          OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76          stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77          single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78          A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79          (tests / non-server) ? behaviour unchanged."""
80          if _cloud_dispatch_possible(config):
81              dep = getattr(config, "deployment", None)
82              return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83          return 1
84
85
86      def configure_drain(config: Any = None) -> None:
87          """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88          server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89          global _drain_max_workers
90          _drain_max_workers = _resolve_drain_workers(config)
91
92
93      def _get_executor() -> ThreadPoolExecutor:
94          """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95          ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96          global _job_executor
97          if _job_executor is None:
98              _job_executor = ThreadPoolExecutor(
99                  max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
```

```
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
```

```
        is_compile = job.get("kind") == "compile"
        try:
            if is_compile:
                result = _main._execute_compile(
                    config,
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
```

```
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main

        ran = 0
        while True:
            job = db.claim_due_job()
```

```
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job, config, db)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting(config, db)
        return ran


    def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
        due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
        import backend.main as _main

        try:
            delay = db.seconds_until_next_due()
        except Exception as e:  # pragma: no cover - defensive; never wedge the worker
            logger.error(f"Could not compute next-due delay: {e}")
            return
        if delay is None:
            return  # nothing waiting
        # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
        _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)


    def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
        """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
        scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
        monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
        import backend.main as _main

        timer = threading.Timer(
            delay_sec,
            lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
        )
        timer.daemon = True
        timer.start()


    #: A parked job further out than this is re-checked by a capped wake rather than one
long
    #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
    _MAX_DRAIN_WAKE_SEC = 60.0


    #
----------------------------------------------------------------------------------------
    # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
    # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
    # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
    # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
```

```
    # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
    # re-dispatching (no double GPU spend).

    #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
    _RESUME_POLL_EVERY_SEC = 15.0


    def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
        """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
        orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
        after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
        thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
        NEVER re-dispatched from here. Returns the number of resumes started.

        On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
        exist ? the rows get today's honest terminal failure instead."""
        rows = db.interrupted_remote_process_jobs()
        if not rows:
            return 0
        if not _cloud_dispatch_possible(config):
            for job in rows:
                db.mark_job_failed(str(job["id"]), "interrupted by server restart")
            logger.warning(
                "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
                "dispatch to Cloud Run, so no remote worker can be in flight.",
                len(rows),
            )
            return 0
        # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
        # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
        # default target itself is cloud_run (on a default-local server with cloud-override
        # capability, a NULL row ran in-process ? no remote worker exists to resume).
        default_is_cloud = (
            getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
        )
        started = 0
        for job in rows:
            # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
            # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
            requested = (str(job.get("compute") or "")).strip().lower()
            if requested != "cloud" and not default_is_cloud:
                db.mark_job_failed(str(job["id"]), "interrupted by server restart")
                continue
            t = threading.Thread(
                target=_resume_one_remote_job,
                args=(config, db, dict(job)),
                daemon=True,
                name=f"resume-{str(job['id'])[:8]}",
            )
            t.start()
            started += 1
        if started:
            logger.warning(
                "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
                started,
```

```
            )
        return started


    def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
        """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
        ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
        fail the job HONESTLY from what the bucket says. Never dispatches."""
        from backend import orchestration, storage
        from backend.infrastructure.execution_policy import (
            ExecutionPolicy,
            PolicyTimeout,
            poll_until,
        )

        job_id = str(job["id"])
        video_id = str(job["video_id"])
        try:
            raw_params = job.get("params")
            params = (
                raw_params
                if isinstance(raw_params, dict)
                else json.loads(raw_params or "{}")
            )
            if params.get("force"):
                # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
                # but a forced job must NEVER complete from the bucket cache ? force
deliberately
                # bypasses it, and nothing proves the report belongs to the forced run
rather
                # than an older completed one.
                db.mark_job_failed(
                    job_id,
                    "control plane restarted during a forced reprocess; the cached result "
                    "cannot be trusted for force=true ? re-submit with force",
                )
                return
            if (str(job.get("compute") or "")).strip().lower() == "local":
                # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
                # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
                db.mark_job_failed(job_id, "interrupted by server restart")
                return
            sc = orchestration._storage_settings(config)
            output_root = ((sc or {}).get("output_root") or "").rstrip("/")
            if not sc or not output_root:
                db.mark_job_failed(
                    job_id,
                    "interrupted by server restart (no output bucket configured to recover
from)",
                )
                return
            report_uri = f"{output_root}/outputs/{video_id}/report.json"
            max_wait = float(
                getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
                or 3900.0
            )
            policy = ExecutionPolicy(
                poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
                op_timeout_sec=120.0,
```

```
438                 max_wait_sec=max_wait,
439             )
440             db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441             try:
442                 poll_until(
443                     lambda: True if storage.exists(report_uri, config=sc) else None,
444                     policy,
445                     label=f"resume-{job_id[:8]}",
446                     logger=logger,
447                     # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448                     on_tick=lambda elapsed: db.set_job_progress(
449                         job_id,
450                         "recovering: waiting for the in-flight cloud run ? "
451                         f"{int(elapsed) // 60} min",
452                     ),
453                 )
454             except PolicyTimeout:
455                 db.mark_job_failed(
456                     job_id,
457                     "control plane restarted mid-run and the cloud worker's result never "
458                     "appeared within the poll budget; re-submit the video (a finished result
"
459                     "re-hydrates instantly)",
460                 )
461                 return
462             payload = orchestration.cached_process_result(
463                 db, config, video_id, str(job.get("video_path") or "")
464             )
465             if payload is not None:
466                 db.mark_job_done(job_id, payload)
467                 logger.warning(
468                     "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469                     job_id,
470                     video_id,
471                 )
472                 return
473             # The report exists but did not realize into a usable result ? surface its
verdict.
474             status = "unreadable"
475             try:
476                 report = orchestration._read_bucket_json(report_uri, sc) or {}
477                 status = str(report.get("status") or "unreadable")
478             except Exception:  # noqa: BLE001 ? the failure message is best-effort
479                 pass
480             db.mark_job_failed(
481                 job_id,
482                 f"cloud worker finished without a usable result after a control-plane
restart "
483                 f"(report status: {status}); re-submit the video",
484             )
485         except Exception as e:  # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486             logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487             try:
488                 db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489             except Exception:  # noqa: BLE001
490                 pass
491
```

17. assistant (2026-07-06T17:27:25.964Z)

This confirms the full control-plane flow. Now let me verify the CloudRunCompute.run_infer done-marker/report interaction and check yolo_hybrid (the parallel hybrid segmenter) to see whether it has the same silent-failure leak but is out of scope, plus check the cmd_infer `_fail(2)`

case where `video_id` may not be defined yet.

18. user (2026-07-06T17:27:27.406Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22      owner-verified on the M7 real run.
23      """
24
25      from __future__ import annotations
26
27      import json
28      import logging
29      import os
30      import tempfile
31      import uuid
32      from abc import ABC, abstractmethod
33      from typing import Any, Callable, List, Optional
34
35      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36      from backend.infrastructure.execution_policy import (
37          DEFAULT_POLICY,
38          ExecutionPolicy,
39          PolicyTimeout,
40          poll_until,
41      )
42      from backend.storage import fetch, get_storage, parse_uri
43
44      LOG = logging.getLogger("compute.cloud_run")
45
46      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
```

```
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-polls)."""

        @abstractmethod
        def cancel_execution(self, execution: str) -> None:
            """Cancel a running execution by the name ``run_job`` returned. Called when the bucket-poll
            times out so the abandoned execution doesn't keep billing the GPU until ``--task-timeout``.
            May raise ? the caller treats every failure as best-effort (the original poll timeout is
            NEVER masked by a cancel error)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test dependency)."""

        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            # A None/empty project would build a bogus "projects/None/locations/..." path (job_path
            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is exercisable
            # in CI (where google-cloud-run is absent) and so mypy narrows project to str for job_path.
            if not project:
                raise RuntimeError(
                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP project id "
```

```
98                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                  )
100             try:
101                 from google.cloud import run_v2  # type: ignore[attr-defined]
102             except Exception as exc:  # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             )  # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
119             return (
120                 getattr(op, "metadata", None)
121                 and getattr(op.metadata, "name", "")
122                 or "execution"
123             )
124
125         def cancel_execution(self, execution: str) -> None:
126             # run_job falls back to the literal string "execution" when the operation
metadata carries
127             # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128             # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
129             # google-cloud-run is absent).
130             if "/executions/" not in (execution or ""):
131                 raise RuntimeError(
132                     f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133                     "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
134                     "one from the operation metadata, so there is nothing to cancel by
name)."
135                 )
136             try:
137                 from google.cloud import run_v2  # type: ignore[attr-defined]
138             except Exception as exc:  # pragma: no cover - real SDK not present in CI
139                 raise RuntimeError(
140                     "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141                     "control plane (it is intentionally NOT a CI/test dependency)."
142                 ) from exc
143             client = (
144                 run_v2.ExecutionsClient()
145             )  # pragma: no cover - exercised only on a real run
146             client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147
148
149     class CloudRunCompute(ComputeBackend):
150         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152         def __init__(
153             self,
```

```
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
```

```
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
```

```
271                     "? rescuing the completed result",
272                     self._job_name,
273                     execution,
274                 )
275                 return late
276             try:
277                 self._client.cancel_execution(execution)
278             except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
279                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
280                 LOG.warning(
281                     "cloud-run: best-effort cancel of execution=%s failed: %s",
282                     execution,
283                     cancel_exc,
284                     exc_info=True,
285                 )
286             else:
287                 cancel_note = "cancel requested"
288                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
289             raise PolicyTimeout(
290                 f"{timeout_exc} [cloud-run job={self._job_name} "
291                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
292                 "state in the GCP console]"
293             ) from timeout_exc
294
295         # --- storage-backed completion signal (bucket-poll) ---
296         def _read_marker(self, done_uri: str) -> Optional[dict]:
297             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298             ref = parse_uri(done_uri)
299             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300                 return None
301             return (
302                 self._read_json(done_uri) or {}
303             )  # empty dict still terminates the poll (present == done)
304
305         def _read_json(self, uri: str) -> dict:
306             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309             try:
310                 ref = parse_uri(uri)
311                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                     dst = os.path.join(td, ref.name or "obj.json")
313                     local = fetch(uri, dst, config=self._storage_config)
314                     with open(local, "r", encoding="utf-8") as f:
315                         data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317             except Exception:
318                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
319                 return {}
320
```

19. assistant (2026-07-06T17:27:35.198Z)

Now I have the full control-plane picture. Let me verify one critical concern: in `cmd_infer`, the `_fail` closure captures `video_id`, `scratch`, `storage_cfg`, `args`. Let me trace whether `video_id` is defined at every `_fail` call site.

Looking at cmd_infer:
- Line 249-251: `scratch` defined

```
- Line 247: `storage_cfg` defined
- Line 260: `video_id = compute_video_id(local_video)` ? defined
- Line 275: `_fail` defined
- All three `_fail` calls (line 312 validation rc2, line 320 unknown-segmenter rc2, line 328
Failure rc3) happen AFTER line 260 where `video_id` is set.

So `video_id` IS defined at every call site. Good. Now let me check the CloudRun `run_infer`
returns `rc` (returncode from marker) ? for a `_fail(3)`, the marker has returncode=3. That
propagates up. Let me verify the git status and check whether there's a subtle issue: the
`_dispatch_remote` path returns `result.returncode`. But wait ? that's the *dispatcher* worker.
The actual detector failure happens in the *dispatched container* which runs cmd_infer inline.
Let me look at the yolo_hybrid parallel segmenter to check scope, and confirm the git diff
matches what's applied.
```

20. user (2026-07-06T17:27:38.918Z)

```
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M backend/worker/__main__.py
 M tests/test_tracknet_hybrid.py
 M tests/test_worker.py
```

21. user (2026-07-06T17:27:39.558Z)

```
347:                return []
459:    def process_video(
471:            return []
482:            return []
```

22. assistant (2026-07-06T17:27:47.332Z)

```
The diff is applied (4 files modified, matching). yolo_hybrid is a parallel segmenter that still
returns bare `[]` on failure paths ? but that is explicitly out of scope (the task is about the
trajectory/WASB detector). Let me verify one thing that matters for correctness: the
`_read_report_json` for the failure path writes `status="failed"`, and confirm the
`emit_indexer_report` in the `finally` block does NOT write a competing report.json to the same
bucket path. Let me check what `emit_indexer_report` writes and where.
```

23. user (2026-07-06T17:27:48.284Z)

```
Found 1 file
backend\reporting\__init__.py
```

24. user (2026-07-06T17:27:52.226Z)

```
29:def emit_indexer_report(
74:        output_dir = (config.get("report") or {}).get("output_dir")
75:        res = rec.write(output_dir=output_dir)
```

25. user (2026-07-06T17:27:55.222Z)

```
29      def emit_indexer_report(
30          *,
31          db,
32          video_id: str,
33          video_path: Optional[str],
34          config: Optional[Dict[str, Any]],
35          val_results: List[Any],
36          val_context: Optional[Dict[str, Any]] = None,
37          segmenter_requested: Optional[str] = None,
38          segmenter_actual: Optional[str] = None,
39          was_reingest: bool = False,
40          segmentation_ran: bool = True,
41      ) -> Optional[Dict[str, str]]:
42          """Harvest + write the report. Returns the written file paths, or None if
```

```
43          reporting is unavailable/disabled/failed. Never raises."""
44          config = config or {}
45          # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
46          # Normalize to a plain nested dict so harvest's dict access works uniformly.
47          if hasattr(config, "model_dump"):
48              try:
49                  config = config.model_dump(mode="json")
50              except Exception:
51                  config = {}
52          # Consume run signals unconditionally (even when reporting is off) so the
53          # process-local store never accumulates entries for finished runs.
54          signals = run_signals.pop(video_id)
55          if not OBSREPORT_AVAILABLE or not report_enabled(config):
56              return None
57          try:
58              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
59              candidates = db.query_candidate_segments(video_id) if db is not None else []
60              rec = harvest.record_run(
61                  video_id=video_id,
62                  video_path=video_path,
63                  config=config,
64                  val_results=val_results,
65                  val_context=val_context,
66                  play_segments=play_segments,
67                  candidates=candidates,
68                  segmenter_requested=segmenter_requested,
69                  segmenter_actual=segmenter_actual,
70                  was_reingest=was_reingest,
71                  segmentation_ran=segmentation_ran,
72                  run_signals=signals,
73              )
74              output_dir = (config.get("report") or {}).get("output_dir")
75              res = rec.write(output_dir=output_dir)
76              return res.paths or None
77          except Exception:  # pragma: no cover - defensive; reporting must never break a run
78              return None
79
```

26. assistant (2026-07-06T17:28:08.755Z)

`emit_indexer_report` writes to a *local* `report.output_dir` (the observability report ? technical_md, customer_md, json), NOT to the bucket `outputs/<video_id>/report.json`. It's a completely separate artifact (a different filename and a local dir, never pushed to `out_uri`). So there's no double-write or competing status. Good ? this confirms the two report.json artifacts don't collide.

Now let me carefully examine one subtle concern flagged in the task: the line `segments = result.value if hasattr(result, 'value') else result`. Since `isinstance(result, Failure)` returns before this line, a Failure can never reach it. But could a `Success` wrapper mis-unwrap? A `Success` has `.value`, so it correctly unwraps. A bare list has no `.value`, so `result` itself is used. Both correct.

But there's a subtle point: `Failure` also has an `unwrap()` but does it have `.value`? Let me re-check `results.py` ? `Failure` has `message`, `details`, `is_recoverable`, and methods `is_success()`, `unwrap()`. It does NOT have a `.value` attribute. So even if a Failure slipped past (it can't), `hasattr(result, "value")` would be False and it'd assign the Failure object to `segments`. But the isinstance check guards this. Fine.

Let me now check the `_run_claimed_job` local-execution path ? the non-cloud in-process path. When `_execute_processing` runs the segmenter locally (not via worker), does the Failure surface? Let me check `_execute_processing` / `_maybe_dispatch_remote` in main.py for the isinstance handling in the LOCAL (non-worker, non-cloud) job path.

27. user (2026-07-06T17:28:10.445Z)

```
905:def process_video(req: ProcessRequest, request: Request):
1552:                result = segmenter.process_video(
1557:                if isinstance(result, Failure):
1562:                    segments = result.value if hasattr(result, "value") else result
```

28. assistant (2026-07-06T17:28:15.597Z)

The only in-process `process_video` call with segmenter handling in main.py is at line 1552 (the CLI path), which correctly handles the Failure. Let me confirm where `_execute_processing` actually runs the segmentation ? it likely delegates to a shared indexing function. Let me find it.

29. user (2026-07-06T17:28:17.526Z)

```
backend\results.py:72:    that forget it (e.g. ``len(process_video(...))``) crash with
``TypeError: ... has no len()``
backend\results.py:75:    if isinstance(result, Failure):
backend\results.py:77:    value = result.value if hasattr(result, "value") else result
backend\worker\__main__.py:322:        result = segmenter_cls(db, config).process_video(
backend\worker\__main__.py:326:        if isinstance(result, Failure):
backend\worker\__main__.py:329:        segments = result.value if hasattr(result, "value") else
result
backend\api\jobs.py:7:per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
backend\api\jobs.py:32:    """ATOMIC idempotent-submit hook for backend.main.process_video (PR
#481 review): delegates to
backend\main.py:905:def process_video(req: ProcessRequest, request: Request):
backend\main.py:1552:                result = segmenter.process_video(
backend\main.py:1557:                if isinstance(result, Failure):
backend\main.py:1562:                    segments = result.value if hasattr(result, "value")
else result
backend\orchestration.py:634:def _execute_processing(
backend\orchestration.py:795:                result = segmenter.process_video(
backend\orchestration.py:806:        if isinstance(result, Failure):
backend\orchestration.py:818:        segments = result.value if hasattr(result, "value") else
result
backend\eval\served_gate_a.py:13:verification reflects what ``process_video`` actually hands to
Gemini.
backend\eval\served_gate_a.py:38:# TrajectoryHybridSegmenter.process_video reads for the served
fusion run).
backend\eval\served_gate_a.py:84:    # call the pure decision seams ? no process_video, no DB,
no provider ? exactly as the
backend\pipeline\segmenters\base.py:44:    def process_video(
backend\pipeline\segmenters\base.py:55:        Use isinstance(result, Failure) (or
result.is_success()) to distinguish
backend\pipeline\segmenters\gemini.py:85:        """Resolve the sport profile, provider, and API
key (the process_video preamble).
backend\pipeline\segmenters\gemini.py:620:    def process_video(
backend\pipeline\segmenters\gemini.py:637:        if isinstance(resolved, Failure):
backend\pipeline\segmenters\motion.py:23:    def process_video(
backend\pipeline\segmenters\motion.py:58:        before returning. This is the detection half of
``process_video`` ? it
backend\eval\windowing.py:7:(``TrajectoryHybridSegmenter.process_video``) windows COARSELY
(``vt=0.02, merge_gap=2.0,
backend\eval\windowing.py:111:    ``TrajectoryHybridSegmenter.process_video`` reads exactly
these three:
backend\eval\windowing.py:164:    Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob
reads exactly, against an
backend\pipeline\segmenters\trajectory_hybrid.py:174:    def process_video(  # type:
ignore[override]
backend\pipeline\segmenters\trajectory_hybrid.py:185:        # (worker.cmd_infer, main.py,
orchestration) branches on ``isinstance(result, Failure)``;
backend\pipeline\segmenters\yolo_hybrid.py:67:        ``process_video`` should early-return
``[]`` (bad sport, missing API key,
backend\pipeline\segmenters\yolo_hybrid.py:459:    def process_video(
```

30. assistant (2026-07-06T17:28:25.669Z)

The critical in-process serving path is `orchestration._execute_processing` at line 634, which calls `process_video` at line 795 and handles the Failure at line 806. Let me read that block carefully ? this is the path the async job worker uses for a LOCAL (non-cloud) run, and it's where the wf294 report.json status is ultimately written for the serving surface.

31. user (2026-07-06T17:28:27.508Z)

```
790             segmenter = segmenter_cls(db, config)
791             try:
792                 # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
793                 # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
794                 with _LOCAL_COMPUTE_LANE:
795                     result = segmenter.process_video(
796                         video_id, local_video, req.player_pool, req.max_frames
797                     )
798             except Exception:
799                 # A raising segmenter must not leave half-written rows: restore the pre-run
800                 # state (same destroy-after-confirm contract as the Failure branch), then
let
801                 # the job worker record the failure.
802                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
803                 raise
804             segmentation_ran = True
805
806             if isinstance(result, Failure):
807                 logger.error(f"[API] Segmenter failed: {result.message}")
808                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
809                 response = {
810                     "video_id": video_id,
811                     "status": "failed",
812                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
813                     "results": [r.model_dump() for r in val_results],
814                     "play_segments": [],
815                 }
816                 return response
817
818             segments = result.value if hasattr(result, "value") else result
819             notify("finalizing")
820             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
821
822             response = {
823                 "video_id": video_id,
824                 "status": "success",
825                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
826                 "results": [r.model_dump() for r in val_results],
827                 "play_segments": segments,
828             }
829             return response
830         finally:
831             # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
832             # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
833             # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
834             # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
835             if isinstance(response, dict):
836                 response.setdefault(
```

```
837                     "compute",
838                     {
839                         "path": compute_actual or "not_run",
840                         "requested": getattr(req, "compute", None),
841                     },
842                 )
843             prov = response.get("compute", {}) if isinstance(response, dict) else {}
844             logger.info(
845                 "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
846                 video_id,
847                 getattr(req, "compute", None),
848                 prov.get("path"),
849                 prov.get("dispatched"),
```

32. assistant (2026-07-06T17:28:40.705Z)

The in-process serving path (`_execute_processing`) correctly handles the Failure at line 806 ? returns `status="failed"`. This response is what `_run_claimed_job` writes via `db.mark_job_done(job_id, result)`. So a Failure in the local job path lands as `status="failed"` too. All three consumer paths (worker.cmd_infer, main.py CLI, orchestration._execute_processing) correctly branch on the Failure.

Now let me verify one thing about the `_dispatch_remote` return in cmd_infer: when `compute is not None`, cmd_infer returns `_dispatch_remote(...)` at line 245, BEFORE `scratch`/`video_id`/`storage_cfg` are defined and before `_fail` exists. That's fine ? the dispatch path returns early and never touches `_fail`. Good.

Now, one important verification: the task asks whether the 0-segment "loud diagnostic" still fires for the LEGIT empty case and NOT for the failure case. In cmd_infer, the warning at line 334 `if not segments:` is inside the `try` block AFTER `status = "success"` (line 330), which is only reached when `result` is NOT a Failure (the Failure branch returns at line 328). So the loud diagnostic fires only for the legit empty case. Correct.

Let me now double-check whether there's an idempotency concern I should flag: in `_fail`, when both a validation failure (rc2) occurs AND `--done-uri` is set, it writes report.json with `status="failed"`. But consider the `_dispatch_remote` codepath more carefully ? actually that's a separate process. Let me now check the test file to confirm the assertions are actually valid against the real code, particularly the new worker test.

33. user (2026-07-06T17:28:42.154Z)

```
225
226
227     def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
228         # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
229         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
230         monkeypatch.setattr(
231             wmain.validator_registry,
232             "run_all_with_context",
233             lambda path, cfg: ([_OKResult()], {}),
234         )
235         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
236         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
237         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
238         video = tmp_path / "in.mp4"
239         video.write_bytes(b"FAKE")
240         out = tmp_path / "bucket"
241
242         # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
243         # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
244         args = wmain.build_parser().parse_args(
245             [
```

```
246                 "infer",
247                 "--video-uri",
248                 str(video),
249                 "--out-uri",
250                 str(out),
251                 "--scratch",
252                 str(tmp_path / "s"),
253                 "--segmenter",
254                 "wasb_hybrid",
255             ]
256         )
257         with caplog.at_level("WARNING", logger="backend.worker"):
258             rc = wmain.cmd_infer(args)
259         assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
260         msgs = " ".join(r.getMessage() for r in caplog.records)
261         assert "0 segments" in msgs and "vidZ" in msgs
262         assert "detector_impl=native" in msgs  # surfaces the resolved detector
263         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
264         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
```

34. assistant (2026-07-06T17:28:54.593Z)

The tests are consistent with the code. Now let me consider a subtle but important correctness angle regarding the `_EmptySeg` and `_FailSeg` test doubles: their `process_video` signatures. `_EmptySeg.process_video(self, video_id, path, max_frames=None)` and `_FailSeg.process_video(self, video_id, path, max_frames=None)`. The worker calls `process_video(video_id, local_video, max_frames=args.max_frames)` ? matches. Good.

Now let me examine one genuinely important concern: the `_FailSeg` test monkeypatches `segmenter_registry.get` to return `_FailSeg`, but the worker also constructs the DB and calls `db.add_video`. That works with a real Database. Let me verify the test doesn't need a real segmenter. It's a monkeypatch, so `segmenter_cls(db, config)` = `_FailSeg(db, config)` whose `__init__(self, db, config)` accepts them. Good.

Now let me look at a potential real defect. In cmd_infer's `_fail`, the failure report.json is written with 0 segments and pushed to the bucket. But consider: the validation-failure path (`_fail(2)`) is reached at line 312, which is INSIDE the `try:` block (line 306). After `_fail(2)` returns, the `finally:` block (line 353) still runs `emit_indexer_report`. That's fine (separate artifact). But note: `_fail` does `return _serialize_and_push_outputs(...)` then `_write_done_marker` then `return rc`. The `return _fail(2)` statement itself is inside the `try`, so the `finally` runs after `_fail` completes but before the function actually returns. Order: `_fail` pushes failure report ? `finally` emits local observability report ? function returns rc. No conflict.

But wait ? there's a subtle issue. When `_fail` is called for validation failure, it pushes report.json to the bucket. Then the `finally` runs `emit_indexer_report`. Then the function returns. That's fine. But is there a scenario where BOTH `_fail` writes a "failed" report AND the success tail (line 369) writes a "success" report to the same path? No ? because every `_fail` call is a `return` statement, so control never reaches the success tail after `_fail`. Confirmed no double-write with conflicting status.

Let me verify the full test suite status claim and run the two most relevant test files to be certain the assertions pass against the real code.

35. user (2026-07-06T17:29:01.716Z)

```
....................................                                     [100%]
40 passed in 2.86s
```

36. assistant (2026-07-06T17:29:04.583Z)

Both modified test files pass (40 tests). Let me run a quick ruff/mypy sanity check on the two modified source files to confirm the constraints hold, and verify the type annotation with the

forward-ref string is well-formed.

37. user (2026-07-06T17:29:10.951Z)

```
All checks passed!
---MYPY---
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

38. assistant (2026-07-06T17:29:14.370Z)

Ruff clean, mypy clean on trajectory_hybrid.py (which is NOT quarantined). Now let me consider the deeper correctness concern about the `# type: ignore[override]` on line 174 combined with the new Union return type. The base class `process_video` declares a `Result` contract. Let me check the base signature to be sure the override-ignore is still needed and correct (not masking a new mismatch).

39. user (2026-07-06T17:29:16.238Z)

```
40              """Human-readable explanation of this segmenter."""
41              pass
42
43          @abstractmethod
44          def process_video(
45              self,
46              video_id: str,
47              video_path: str,
48              player_pool: List[str] = None,
49              max_frames: int = None,
50          ) -> "Result[List[Dict[str, Any]]]":
51              """
52              Processes a video to detect play segments, populates SQLite database,
53              and returns a Result.
54
55              Use isinstance(result, Failure) (or result.is_success()) to distinguish
56              "no segments found" from "something went wrong" (missing key, healthcheck
57              failure, etc.). This is the start of the standardized error/result contract.
58              """
59              pass
60
61
62      class SegmenterRegistry:
63          def __init__(self):
64              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
65
66          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
67              """Registers a new segmenter class under its name property."""
68              try:
69                  temp_inst = segmenter_cls(None, {})
```

40. assistant (2026-07-06T17:29:32.439Z)

The base declares `Result[List[Dict[str, Any]]]` = `Union[Success[List[...]], Failure]`. The override now returns `Union[List[Dict[str, Any]], Failure]` ? the bare `List` arm is still a contract-mismatch vs `Success[List]`, which is exactly why the `# type: ignore[override]` remains necessary and correct. mypy confirms clean. The annotation change is well-formed and the ignore is still accurate.

Now let me do a thorough trace of one final edge case the task explicitly raised: could a `Success` wrapper mis-unwrap or a bare list mis-unwrap at line 329? The trajectory segmenter never returns a `Success` ? it returns either a `Failure` or a bare list. So in practice

`result` is `Failure` (caught) or bare list (? `hasattr(result, "value")` is False ? `segments = result`). Correct. Even if a hypothetical future segmenter returned `Success`, the `.value` unwrap handles it. No mis-unwrap.

Let me verify there isn't a subtle bug where `_serialize_and_push_outputs` in `_fail` could raise because `scratch` outputs dir already contains a partially-written file from a prior partial run within the same process. It does `os.makedirs(out_local, exist_ok=True)` and overwrites both files. No issue.

One more: let me confirm the `_write_report_json` writes `segments: []` for the failure case and that the test asserting `report["segments"] == []` holds ? yes, `_serialize_and_push_outputs(..., [], "failed", ...)` passes an empty list, and `_write_report_json` builds `"segments": []`. Correct.

Let me now examine whether there's any path where the WASB detector's `run_predict` timeout produces a truthy but empty CSV path (not `None`/`""`), which would slip past the `if not csv_path:` guard and proceed to `parse_trajectory_csv` yielding zero points ? zero windows ? bare `[]` "success". This would be a remaining leak. Let me check what `run_predict` returns on a timeout.

41. user (2026-07-06T17:29:34.568Z)

```
backend\pipeline\detectors\native_wasb_runner.py:81:    timeout_sec: int = 1800  # 30 min; a
hung GPU call is killed (0 = no timeout)
backend\pipeline\detectors\native_wasb_runner.py:113:            timeout_sec=w.timeout_sec,
backend\pipeline\detectors\native_wasb_runner.py:140:            timeout=(self.cfg.timeout_sec
or None),
backend\pipeline\detectors\native_wasb_runner.py:146:        (missing python / timeout / import
error) is treated as 'no CUDA'."""
backend\pipeline\detectors\native_wasb_runner.py:149:        except (FileNotFoundError,
subprocess.TimeoutExpired):
backend\pipeline\detectors\native_wasb_runner.py:213:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\native_wasb_runner.py:236:    def run_predict(
backend\pipeline\detectors\native_wasb_runner.py:256:            return None
backend\pipeline\detectors\native_wasb_runner.py:305:            return None
backend\pipeline\detectors\native_wasb_runner.py:306:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\native_wasb_runner.py:308:                "WASB native inference
timed out after %ss.", self.cfg.timeout_sec
backend\pipeline\detectors\native_wasb_runner.py:310:            return None
backend\pipeline\detectors\native_wasb_runner.py:316:            return None
backend\pipeline\detectors\native_wasb_runner.py:321:            return None
backend\pipeline\detectors\native_wasb_runner.py:334:        return out_csv
backend\pipeline\detectors\base.py:105:    and ``timeout_sec`` attributes (TrackNetConfig and
WasbConfig both qualify).
backend\pipeline\detectors\base.py:106:    ``timeout_sec == 0`` maps to None = wait forever
(configs default 1800s).
backend\pipeline\detectors\base.py:116:            argv, capture_output=True, text=True,
timeout=(self.cfg.timeout_sec or None)
backend\pipeline\detectors\base.py:130:        except (subprocess.TimeoutExpired, OSError):
backend\pipeline\detectors\base.py:131:            return None
backend\pipeline\detectors\base.py:135:            return None
backend\pipeline\detectors\base.py:151:            return None
backend\pipeline\detectors\base.py:154:            return None
backend\pipeline\detectors\base.py:158:            return None
backend\pipeline\detectors\base.py:173:        except (subprocess.TimeoutExpired, OSError) as e:
backend\pipeline\detectors\base.py:189:        except (subprocess.TimeoutExpired, OSError):
backend\pipeline\detectors\base.py:190:            return None
backend\pipeline\detectors\base.py:192:            return None
backend\pipeline\detectors\base.py:204:    def run_predict(
backend\pipeline\detectors\court_gate.py:132:    return out
backend\pipeline\detectors\rally_gate.py:128:    return out
backend\pipeline\detectors\stub_runner.py:111:    def run_predict(
backend\pipeline\detectors\stub_runner.py:129:            return out_csv
backend\pipeline\detectors\stub_runner.py:140:        return out_csv
backend\pipeline\detectors\stub_runner.py:188:    return out_csv
```

```
backend\pipeline\detectors\tracknet_runner.py:47:    timeout_sec: int = 1800  # 30 min; kills a
hung call (0 = no timeout)
backend\pipeline\detectors\tracknet_runner.py:65:           timeout_sec=tn.timeout_sec,
backend\pipeline\detectors\tracknet_runner.py:122:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\tracknet_runner.py:131:    def run_predict(
backend\pipeline\detectors\tracknet_runner.py:147:            return None
backend\pipeline\detectors\tracknet_runner.py:163:            return None
backend\pipeline\detectors\tracknet_runner.py:176:                return None
backend\pipeline\detectors\tracknet_runner.py:190:            return None
backend\pipeline\detectors\tracknet_runner.py:204:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\tracknet_runner.py:206:                "TrackNet inference timed out
after %ss.", self.cfg.timeout_sec
backend\pipeline\detectors\tracknet_runner.py:208:            return None
backend\pipeline\detectors\tracknet_runner.py:214:            return None
backend\pipeline\detectors\tracknet_runner.py:220:            return None
backend\pipeline\detectors\tracknet_runner.py:243:            return None
backend\pipeline\detectors\tracknet_runner.py:248:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\tracknet_runner.py:250:            return None
backend\pipeline\detectors\tracknet_runner.py:255:            return None
backend\pipeline\detectors\tracknet_runner.py:545:        return out
backend\pipeline\detectors\tracknet_runner.py:596:        return out, flags
backend\pipeline\detectors\wasb_infer.py:137:        return None
backend\pipeline\detectors\wasb_infer.py:142:        return None
backend\pipeline\detectors\wasb_infer.py:696:    return out_csv
backend\pipeline\detectors\wasb_runner.py:50:    timeout_sec: int = 1800  # 30 min; a hung
GPU/conda call is killed (0 = no timeout)
backend\pipeline\detectors\wasb_runner.py:74:            timeout_sec=w.timeout_sec,
backend\pipeline\detectors\wasb_runner.py:96:        except subprocess.TimeoutExpired:

[Showing results with pagination = limit: 60]
```

42. assistant (2026-07-06T17:29:39.114Z)

This is the crucial confirmation. In `native_wasb_runner.py` line 306-310: on `subprocess.TimeoutExpired` (the 1800s WASB timeout ? the exact wf294 scenario), `run_predict` returns `None`. So in `trajectory_hybrid.process_video`, `csv_path` is `None` ? `if not csv_path:` is True ? returns `Failure`. The timeout path is correctly caught by the new Failure return.

Let me verify line 305-321 of native_wasb_runner to be certain the timeout branch returns None (not a truthy path where the CSV exists but is empty).

43. user (2026-07-06T17:29:40.909Z)

```
288             if self.cfg.batch_size:
289                 argv += ["--batch-size", str(self.cfg.batch_size)]
290             if self.cfg.prefetch_batches:
291                 argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293             logger.info(
294                 "Running native WASB inference (device=%s%s) on %s ...",
295                 device,
296                 " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297                 os.path.basename(norm),
298             )
299             try:
300                 res = self._run(argv, cwd=self._repo_src())
301             except FileNotFoundError:
302                 logger.error(
303                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304                 )
305                 return None
306             except subprocess.TimeoutExpired:
307                 logger.error(
308                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
```

```
309                 )
310                 return None
311             if res.returncode != 0:
312                 logger.error(
313                     "WASB native inference failed:\n%s",
314                     (res.stderr or res.stdout or "")[-2000:],
315                 )
316                 return None
317             if not os.path.exists(out_csv):
318                 logger.error(
319                     "WASB native finished but expected CSV not found at %s", out_csv
320                 )
321                 return None
322             logger.info("WASB trajectory CSV: %s", out_csv)
323
324             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327             if not self.cfg.keep_frames and frames_dir:
328                 logger.info(
329                     "Cache hygiene: removing frame cache for %s "
330                     "(set indexing.wasb.keep_frames=true to retain).",
331                     key,
332                 )
333                 self._rm_rf(frames_dir)
334             return out_csv
335
```

44. assistant (2026-07-06T17:29:56.959Z)

Confirmed. Every WASB failure mode (timeout at 306, non-zero rc at 311, missing CSV at 317, python not found at 301) returns `None` from `run_predict`. In `process_video`, `None` ? `if not csv_path:` True ? returns `Failure`. The wf294 timeout is now correctly surfaced.

I've now traced every return path of `process_video` and every branch of `cmd_infer`, plus all three consumer sites (worker, main.py CLI, orchestration serving path) and the control-plane recovery. Let me do a final holistic assessment of the complete picture.

Full trace summary:

`trajectory_hybrid.process_video` return paths:
1. Healthcheck not ready ? `Failure` ? (was `[]`)
2. `run_predict` returns falsy (timeout/abort/missing CSV) ? `Failure` ? (was `[]`)
3. `if not all_candidate_windows: return []` ? bare `[]` (legit empty) ? correctly stays bare list
4. Tail `return segments_data` ? bare list ?

`cmd_infer` branches:
- `compute is not None` ? `_dispatch_remote` (early return, doesn't touch `_fail`) ?
- validation failure ? `_fail(2)` (video_id defined at line 260) ?
- unknown segmenter ? `_fail(2)` (video_id defined) ?
- `isinstance(result, Failure)` ? `_fail(3)` ? ? the wf294 path
- `segments = result.value if hasattr(...) else result` ? unreachable by Failure (guarded) ?
- 0-segment loud diagnostic ? only fires on non-Failure path (after `status="success"`) ?
- success tail ? `_serialize_and_push_outputs(..., "success")` + done-marker rc=0 ?

`_fail` closure captures `scratch`, `args`, `video_id`, `storage_cfg` ? all defined before `_fail` is defined and before every call site ?. Best-effort push wrapped in try/except ?. Writes `status="failed"` report + done-marker rc/status="failed" ?.

Consumers of the bucket `report.json`:
- `orchestration.probe_process_cache` (line 280): `report.get("status") == "success"` ? a

"failed" report is a miss ?
- `orchestration.cached_process_result` (line 326): `report.get("status") != "success"` ?
returns None ?
- `job_worker._resume_one_remote_job`: waits for report to EXIST, then `cached_process_result`
returns None for a failed report ? `mark_job_failed` with the report's status ?
- `cloud_run.run_infer`: reads done-marker `returncode` (3 for a Failure) ?

`emit_indexer_report` writes a *separate* local observability artifact
(technical_md/customer_md/json to `report.output_dir`), NOT the bucket
`outputs/<id>/report.json` ? no double-write or status conflict ?.

No remaining leak into `status="success"` for a detector timeout/abort. The one bare-`[]` path
(zero windows) is a genuine 0-rally success by design, and it's only reachable AFTER the
detector successfully produced and parsed a trajectory. The fix is correct and complete.

I found no concrete correctness defects. Let me report the empty findings.

45. user (2026-07-06T17:29:57.214Z)

Structured output provided successfully